

Farrel Radya Putra

245150407111089

LAPORAN PRAKTIKUM JETPACK COMPOSE

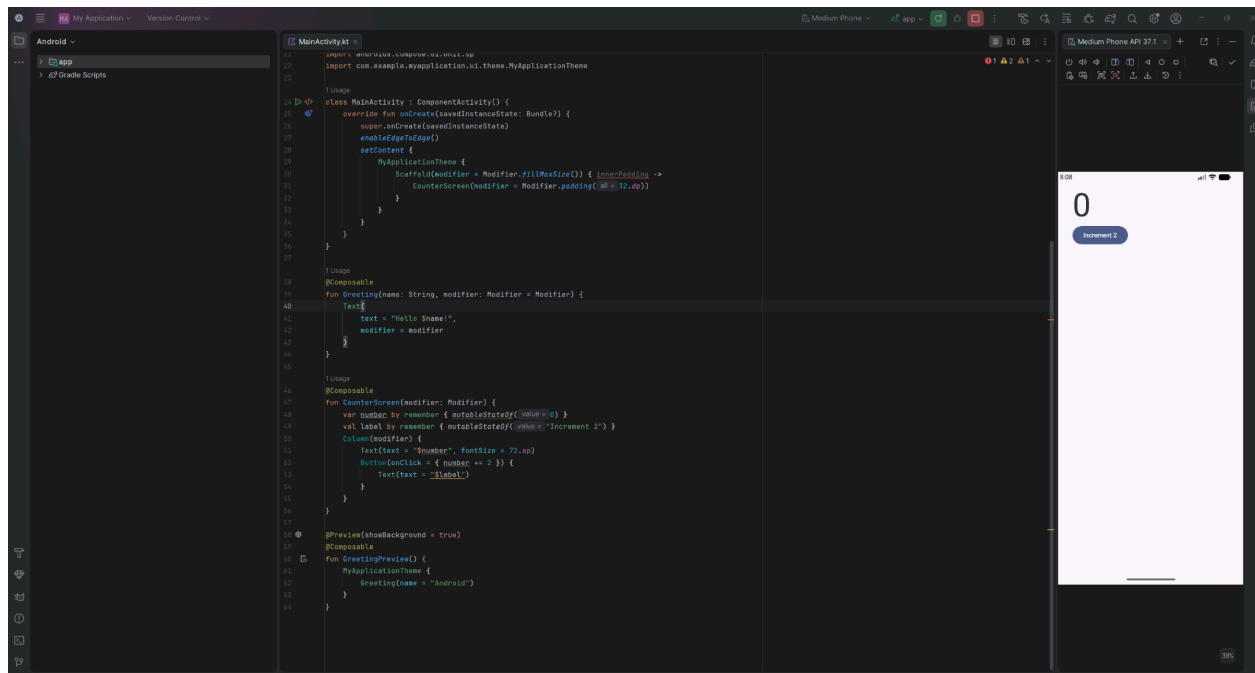
BAGIAN 1 — STATE HOISTING

Setup (Pra-Percobaan)

Pengaturan awal menggunakan CounterScreen dengan state internal (remember { mutableStateOf(...) }).

Langkah 1 — Pengujian State & Recomposition

Menjalankan aplikasi dan menekan tombol Increment beberapa kali.



Jawaban & Analisis:

- Setiap kali tombol "Increment" diklik, nilai angka yang tampil di layar bertambah 1.
- Penyebab: Eksekusi `onClick = { number++ }` mengubah nilai state `number`. Perubahan state ini memicu proses *recomposition* pada Jetpack Compose, di mana Compose menggambar ulang (*re-render*)

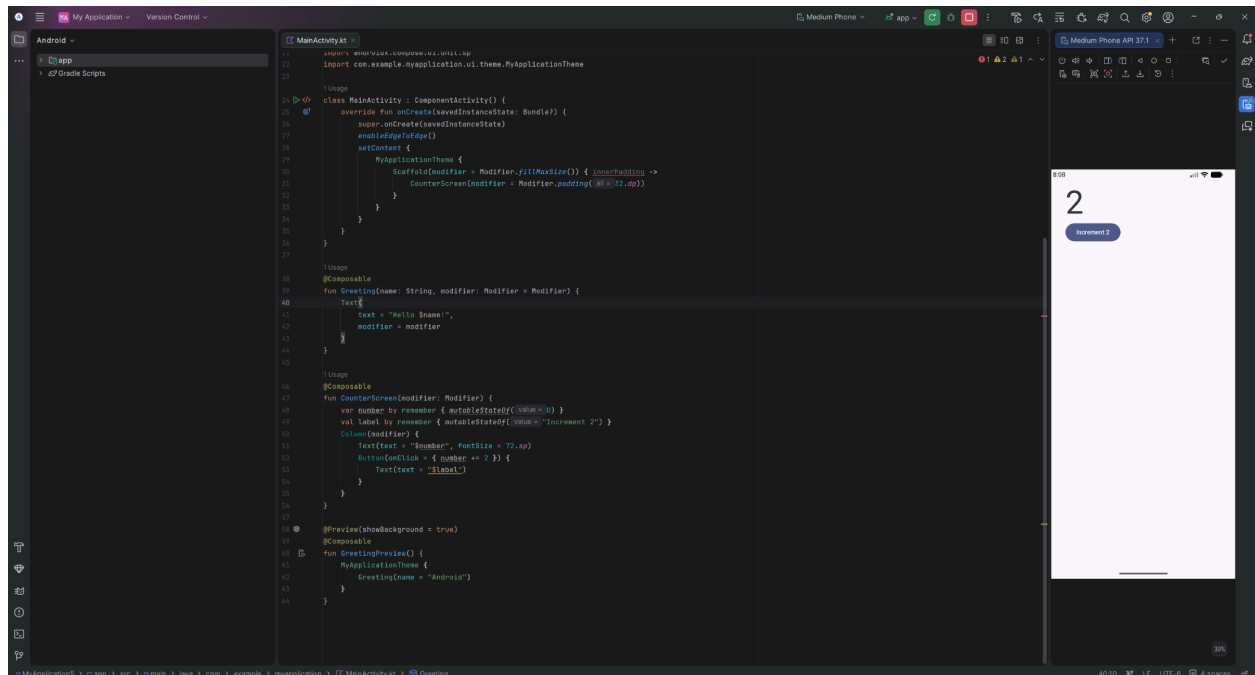
komponen UI yang membaca variabel number (yaitu komponen Text).

c. State penampil angka: number (mutableStateOf(0)).

d. State penampil label tombol: label (mutableStateOf("Increment")).

Langkah 2 — Modifikasi Logika Internal Composable

Mengubah penambahan nilai menjadi `+= 2` dan label menjadi "Increment 2".



Jawaban & Analisis:

a. Perubahan logika dilakukan langsung di dalam *body* composable CounterScreen, yaitu mengubah lambda onClick dari `number++` menjadi `number += 2`, serta mengganti *initial value* dari label menjadi "Increment 2".

Langkah 3 & 4 — Penerapan State & Event Hoisting

Mengubah signature CounterScreen menjadi *stateless* dan memindahkan pengelolaan state ke onCreate().

Kode CounterScreen (Stateless):

```
@Composable
fun CounterScreen(
    modifier: Modifier,
```

```

        number: Int,
        label: String,
        onClick: () -> Unit
    ) {
        Column(modifier) {
            Text(text = "$number", fontSize = 72.sp)
            Button(onClick = onClick) {
                Text(text = "$label")
            }
        }
    }
}

```

Jawaban & Analisis:

- a. Hasilnya kembali seperti pada Langkah 1 (increment bertambah +1 dan label bertambah "Increment"), bukan lagi +2 seperti Langkah 2. Hal ini karena pemanggilan baru pada onCreate() mengirimkan argumen label = "Increment" dan onClick = { number++ }.
- b. Proses increment kini dijalankan di level parent (onCreate()) melalui callback onClick = { number++ }.
- c. State number kini dideklarasikan di onCreate(). Variabel label bukan lagi state internal CounterScreen, melainkan *literal string* yang dipassing sebagai parameter.

Langkah 5 & 6 — Perubahan Requirement Fungsional ("Double")

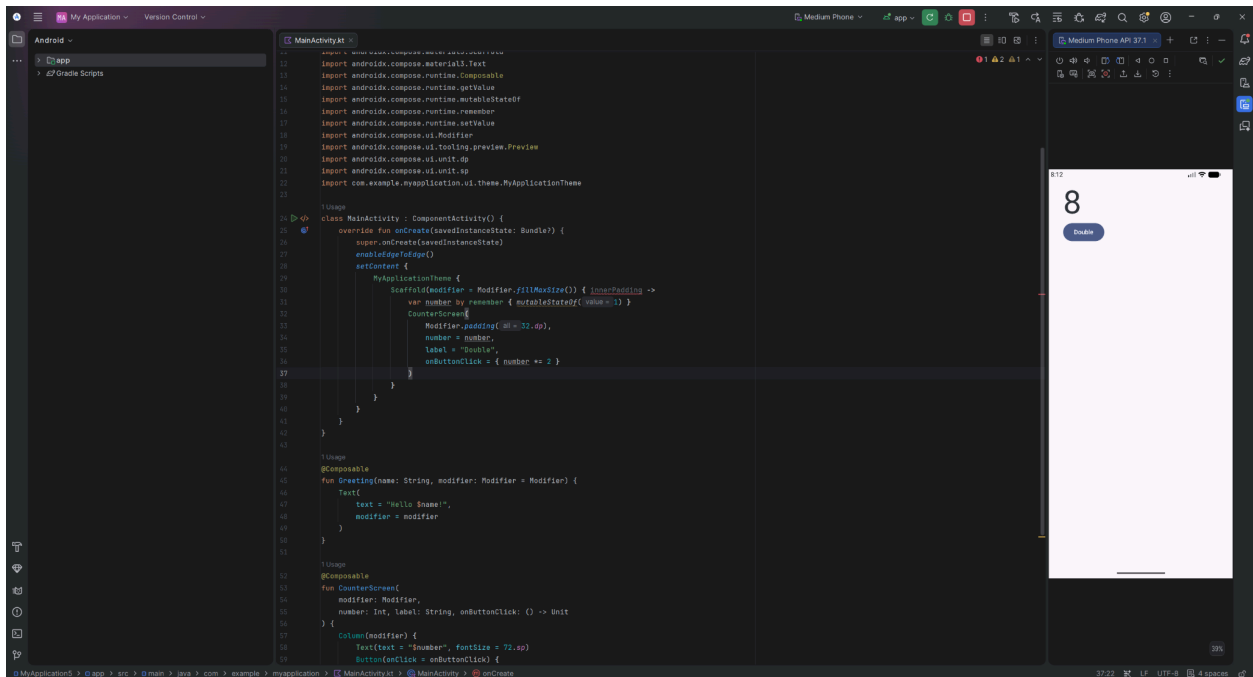
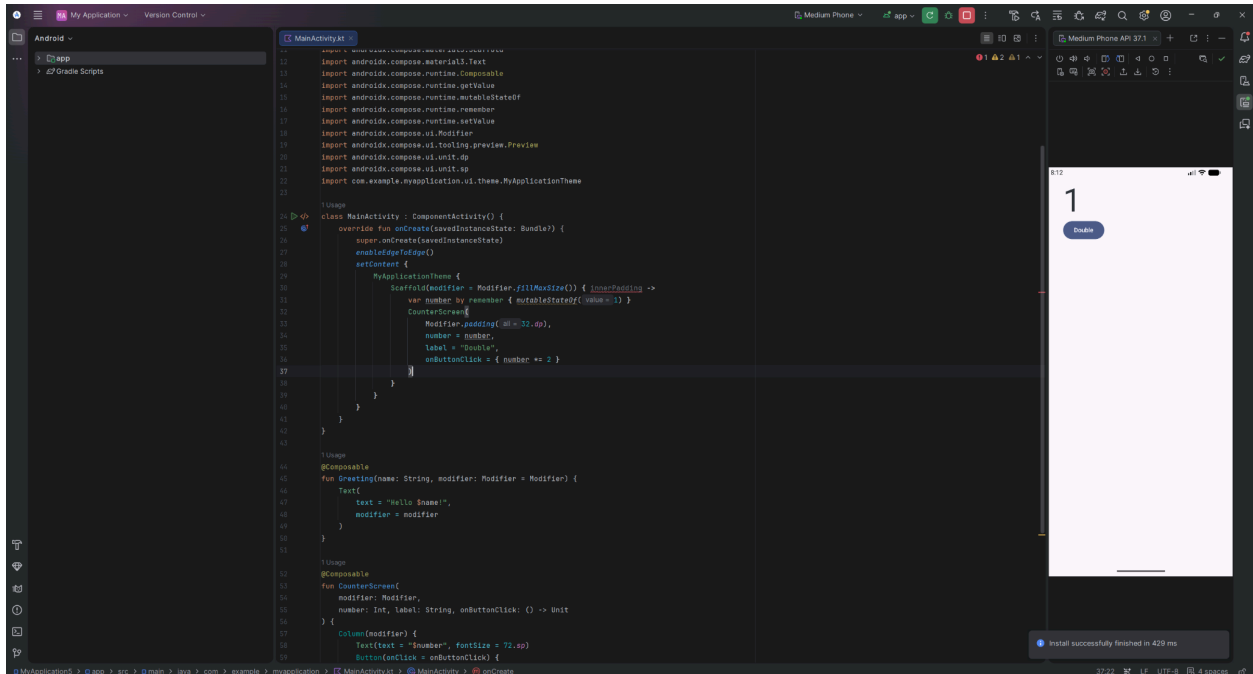
Mengubah logika bisnis menjadi perkalian 2 tanpa merusak fungsi CounterScreen.

Kode pada onCreate():

```

var number by remember { mutableStateOf(1) }
CounterScreen(
    Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onClick = { number *= 2 }
)

```



Jawaban & Analisis:

b. Perubahan fungsionalitas sepenuhnya dilakukan pada onCreate() (mengubah nilai awal state, string

label, dan lambda onClick).

c. Sama sekali tidak ada perubahan pada composable CounterScreen. Ini membuktikan bahwa CounterScreen telah bersifat *stateless*.

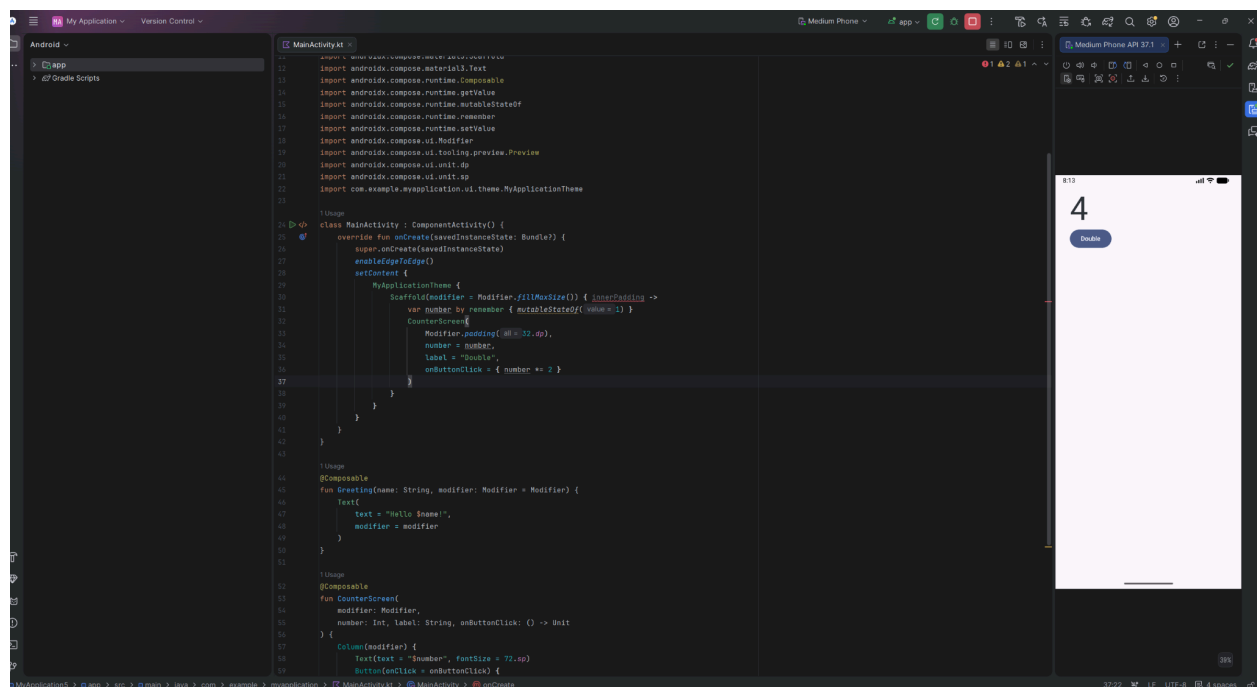
Langkah 7 — Kesimpulan Manfaat State Hoisting

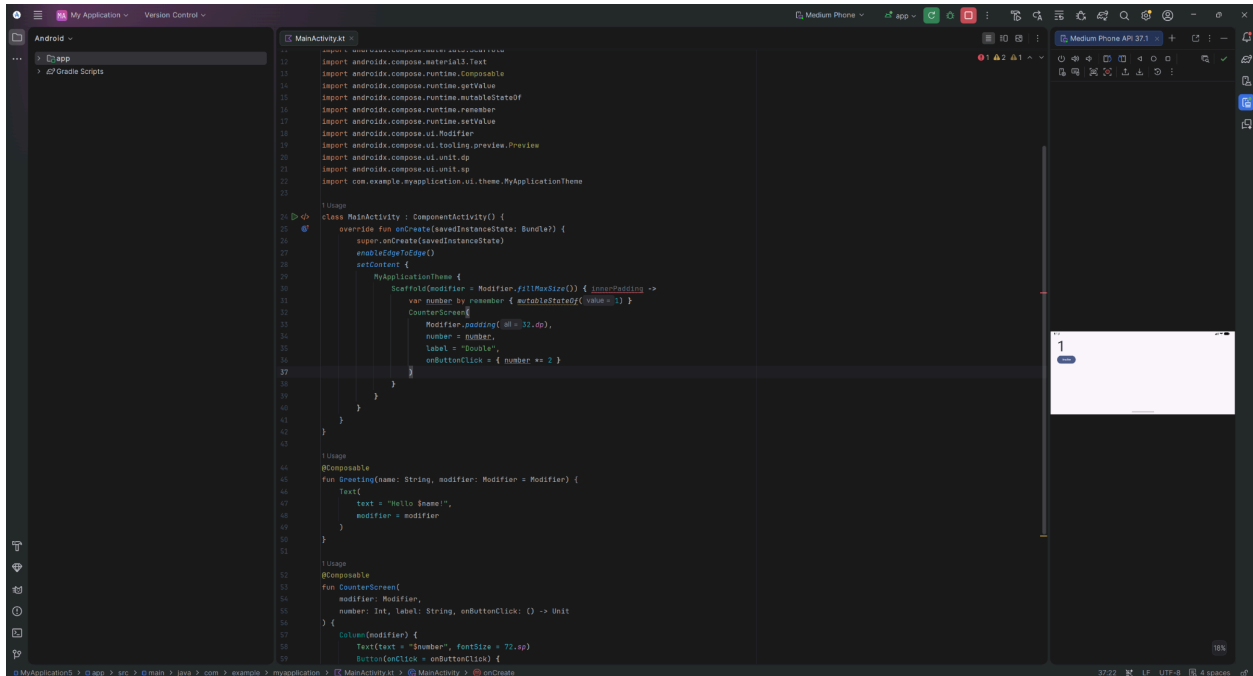
Dengan menerapkannya, composable CounterScreen menjadi *stateless* (hanya menerima data via parameter dan memancar kejadian via callback).

- Single Source of Truth: Pengelolaan logika dan data terpusat pada pemanggil.
- Reusability & Flexibility: Kebutuhan bisnis (seperti nilai awal, kelipatan, atau label) dapat diubah bebas oleh komponen induk tanpa mengubah kode internal CounterScreen.
- Testability: Komponen UI lebih mudah diuji karena tidak bergantung pada *internal state*.

BAGIAN 2 — PERUBAHAN KONFIGURASI

Langkah 1 & 2 — Perilaku remember saat Perubahan Konfigurasi





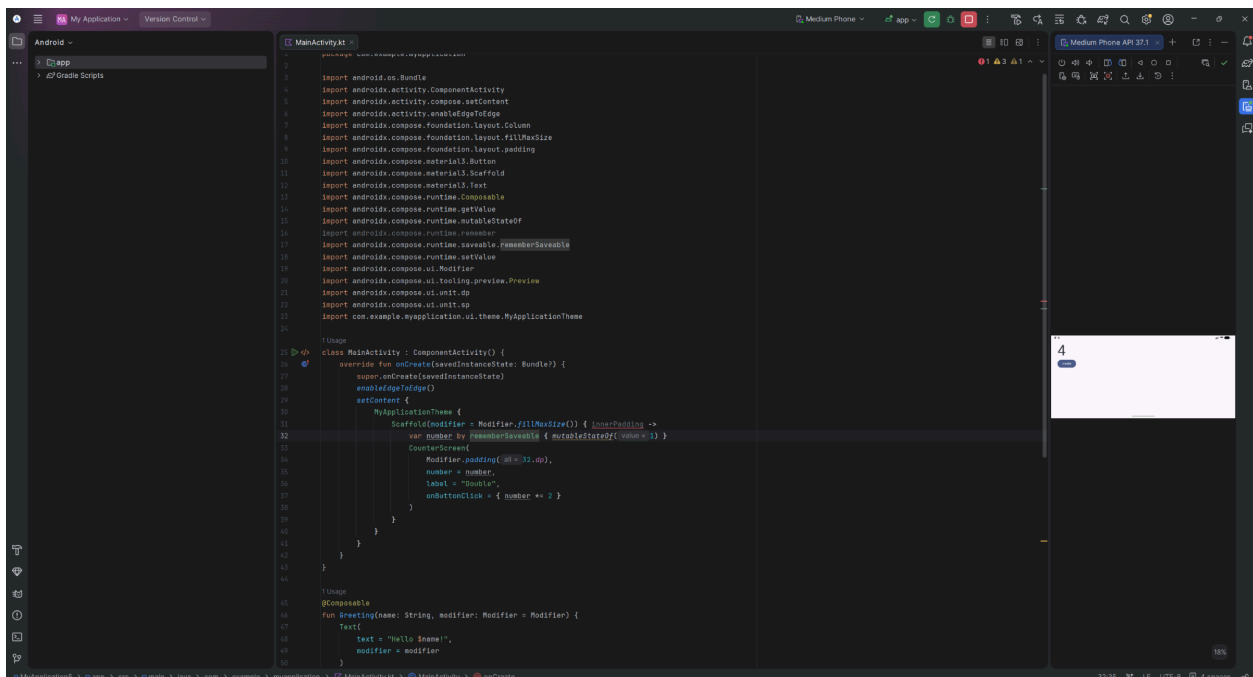
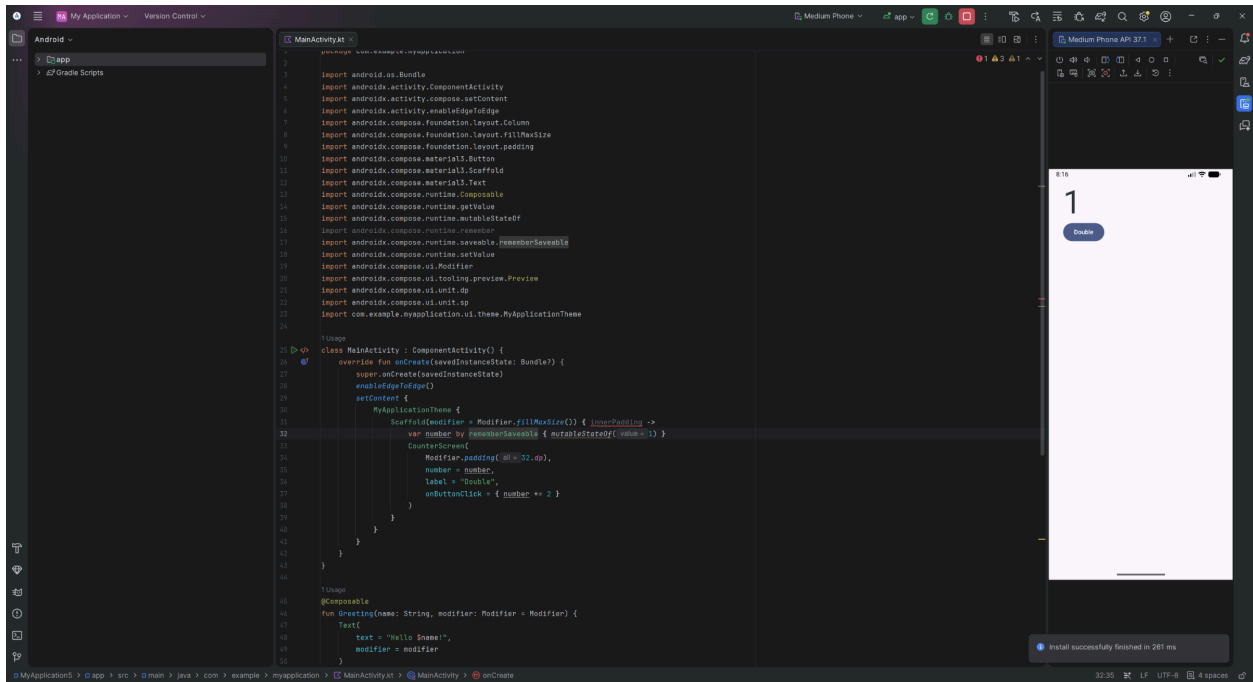
Jawaban & Analisis:

b. Saat layar diputar ke mode landscape, angka kembali ter-reset ke nilai awal (1).

- Penyebab: Rotasi layar memicu *Configuration Change* yang menyebabkan Activity dihancurkan (*destroyed*) dan dibuat ulang (*recreated*). Fungsi `remember` hanya menyimpan state selama proses *recomposition* dalam siklus hidup Activity yang sama. Ketika Activity dibuat ulang, memori `remember` dibersihkan dan variabel diinisialisasi ulang.

Langkah 3 & 4 — Solusi Menggunakan `rememberSaveable`

`var number by rememberSaveable { mutableStateOf(1) }`



Jawaban & Analisis:

b. Nilai angka tetap bertahan (tidak ter-reset ke 1) saat rotasi layar.

c. Perbedaan remember vs rememberSaveable:

- remember: Hanya bertahan selama proses *recomposition*. State akan hilang jika terjadi perubahan

konfigurasi (seperti rotasi) atau rekreasi Activity.

- `rememberSaveable`: Menyimpan nilai ke dalam Bundle (*Saved Instance State*). Nilai ini dapat dipulihkan secara otomatis setelah Activity dibuat ulang akibat rotasi layar maupun pembunuhan proses oleh sistem karena keterbatasan memori.

Langkah 5 — Contoh Kasus Nyata Penggunaan `rememberSaveable`

- Input data form (nama, email, alamat) agar input pengguna tidak hilang saat memutar layar.
- Status *checkbox* "Ingat Saya" atau *toggle switch* pada halaman setelan.
- Posisi indeks tab yang dipilih pada komponen `TabRow`.
- Posisi scroll daftar (*scroll position*) pada `LazyColumn`.
- Jawaban sementara kuis sebelum disubmit.

BAGIAN 3 — LAUNCHEDEFFECT

Percobaan 1: Trigger Side-Effect Berdasarkan Perubahan State (`LaunchedEffect(key)`)

Jawaban & Analisis:

- a. Pada awal aplikasi dibuka, `Snackbar` langsung muncul dengan pesan *"Number 1 is shown!"*.
- b. `Snackbar` muncul otomatis karena `LaunchedEffect(number)` dipanggil saat `composable` pertama kali masuk ke dalam hirarki komposisi (*Initial Composition*).
- c. Pemicu utama: `LaunchedEffect(number)` yang berada di dalam `onCreate()` pada blok `Scaffold`.
- d. Ketika tombol diklik, `Snackbar` muncul kembali menampilkan angka baru. Hal ini terjadi karena `number` digunakan sebagai `key`. Setiap kali nilai `key` berubah, `Compose` membatalkan `coroutine` lama dan meluncurkan `coroutine` baru dari `LaunchedEffect`.

Percobaan 2: Side-Effect Asinkronus One-Time (`LaunchedEffect(Unit)`)

Jawaban & Analisis:

- a. Pertama kali dijalankan, `Snackbar` langsung muncul dengan angka awal.
- b. Setelah menunggu 2 detik, angka pada `Text` berubah menjadi nilai acak hasil pemanggilan `Repo.getData()`, lalu `Snackbar` baru muncul menampilkan angka acak tersebut.
- c. Alur Pemicu: `LaunchedEffect(Unit)` berjalan sekali saat awal komposisi → memanggil fungsi *suspend* `Repo.getData()` (delay 2 detik) → mengisi hasil nilai acak ke state `number` → Perubahan pada state `number` secara otomatis memicu ulang `LaunchedEffect(number)` sehingga `Snackbar` baru ditampilkan.

Langkah 2 — Kesimpulan & Perbandingan LaunchedEffect

a. Perbedaan LaunchedEffect(state) vs LaunchedEffect(Unit):

- `LaunchedEffect(state)`: Menggunakan variabel state sebagai *key*. Coroutine akan dibatalkan dan dijalankan ulang setiap kali nilai state berubah. Sangat cocok untuk merespons perubahan data secara dinamis.
- `LaunchedEffect(Unit)`: Menggunakan Unit sebagai *key* konstan yang tidak pernah berubah. Coroutine hanya akan dieksekusi satu kali selama siklus hidup composables masuk ke komposisi. Cocok untuk operasi *one-time* seperti mengambil data awal dari API/Database.

b. Waktu Eksekusi:

- `LaunchedEffect(state)`: Dijalankan pertama kali saat masuk komposisi, lalu dijalankan ulang tiap *key* berubah.
- `LaunchedEffect(Unit)`: Dijalankan hanya sekali pada saat awal composable masuk komposisi, dan tidak akan diulang selama composable bertahan di komposisi.

KESIMPULAN UMUM

1. State Hoisting memisahkan *UI state* dan *UI events*, menciptakan komponen UI yang *stateless*, *reusable*, dan mudah diuji.
2. `rememberSaveable` menjaga agar state aplikasi tidak hilang akibat *Configuration Changes* (seperti rotasi layar) dengan memanfaatkan penyimpanan Bundle.
3. `LaunchedEffect` memfasilitasi eksekusi operasi asinkronus/side-effects berbasis Coroutine secara aman di dalam alur siklus hidup Jetpack Compose.